COMMUNITY OVER/CODE

AI Plumbing

Orchestrating Local LLMs with Apache Airflow

Oluwafemi Olawoyin
Data & AI Engineer



OCTOBER 11-14, 2026 *GLASGOW, SCOTLAND*

What we'll walk through

01

The tools, and what breaks

why local infra fails differently

02

How Airflow helps

readiness, a pool, cleanup, a gate

03

One night, end to end

the nightly eval run, and a demo

04

Insights

patterns for any exclusive workload

I build local AI tools that run on the user's hardware.

intentKeeper
flags manipulation in feeds

empathySync
helps users need it less

// both open source - nothing leaves the machine, no cloud, no telemetry.

Airflow doesn't run these tools.

It grades them every night, and tells me when one slips.

```
# it grades the models, a human checks the scores, then it files what it found.
```

SECTION 01

Keeping local models running.

// the plumbing

The first version was a shell script.

```
# crontab:  0 3 * * *  run-evals.sh >> /tmp/evals.log 2>&1
nohup ollama serve &
sleep 30                          # hope it's loaded by now
curl -s localhost:11434 || sleep 30    # "readiness"
python run_evals.py               # 3am OOM: no traceback, no retry
```

it kept failing at night. I'd find out the next morning.

Most failures in local AI are infrastructure failures.

```
// process lifecycle, resource contention, observability - Airflow has patterns.
```

FAILURES THAT ONLY EXIST LOCALLY

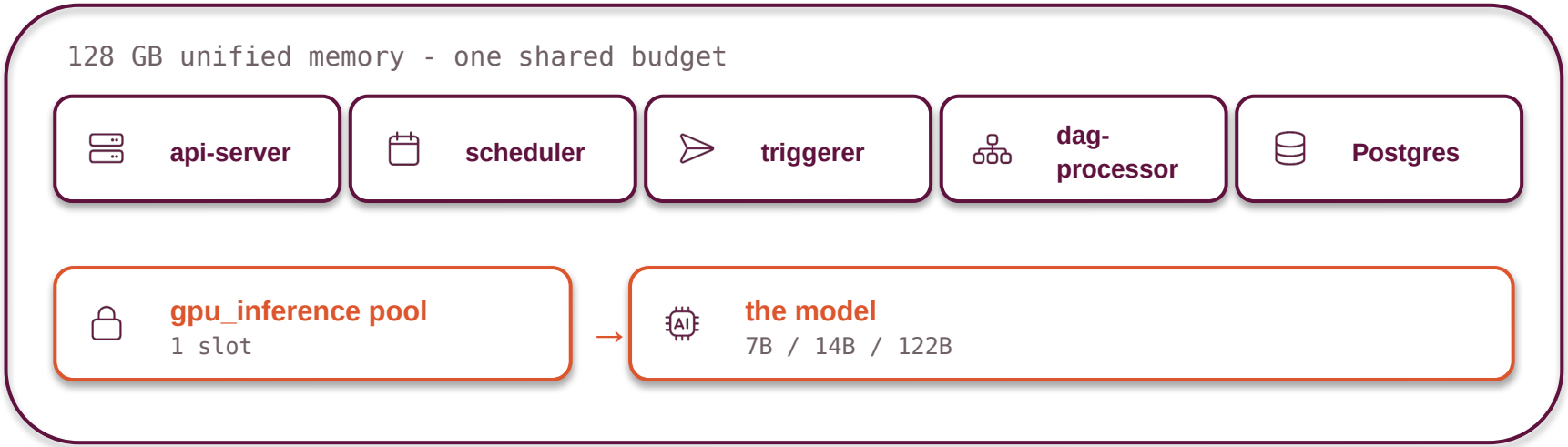
discrete GPU · RTX 4070 · 12 GB

- CUDA OOM
- corrupted pull
- zombie process
- disk full

unified memory · GB10 · 128 GB

- OOM-killer
- GPU vs Postgres for RAM
- aarch64 wheel gaps
- USB automount races

One machine. Everything is a tenant.



one door in front of the model - one big load at a time.

SECTION 02

How Airflow helps.

// familiar tools, a workload that misbehaves

I didn't build a framework. I used one.



// nothing new here - each familiar primitive kills a failure that only exists on the box.

Confirm it's there without loading it.

```
@task.sensor(poke_interval=60, mode="reschedule")
def ollama_ready(models):
    have = list_models("http://localhost:11434")
    return all(m in have for m in models)
```

```
# @task.sensor, reschedule mode - it frees the worker between checks.
```

One model set at a time.

```
# one model set resident at a time
airflow pools set gpu_inference 1

@task(pool="gpu_inference")
def score_model(model): ...
```

// one eval's set at a time - the next set only loads once this one is freed.

A leaked model kills the next run.

```
        @task(trigger_rule="all_done")
        def cleanup():
            unload(MODEL)   # keep_alive = 0, free the memory
```

```
# trigger_rule=all_done runs the unload whatever happened upstream.
```

Two kinds of failure, two different fixes.

- Bad output that won't fit the schema: output_retries re-prompts the model
- Bad infra - a 503 or a timeout: retries re-run the task

```
# output-retries live in the eval (Inspect); task-retries are Airflow.
```

The biggest model my evals run, on the same DAGs.

14B → 120B

Not faster - unified memory trades bandwidth for room.

It just fits - the big model runs where it couldn't before.

I moved the whole stack. The DAGs didn't change.

Rebuilt	Unchanged
every venv	**every DAG**
every container image	**the pool pattern**
Postgres (dump / restore)	**sensors, retries, alerts**

```
# Ryzen + RTX 4070 to ARM GB10 - the orchestration layer stayed put.

# next tenant to test: Jev, a new decision model for the classifier - same DAGs.
```

The orchestration is the durable part.

The model and the box are tenants you swap underneath it.

```
// that's the whole talk - everything else is a tenant.
```

The night stops and waits for a person.



The run scores every model, then pauses.

It waits for a review before it files anything.

No response in 24 hours: it files nothing and closes clean.

```
// the model grades; the person decides. Airflow 3 makes that a first-class step.
```

On local hardware, the lifecycle layer is empty.

what keeps a local model alive:

health · lifecycle (pull / load / unload) · a memory pool

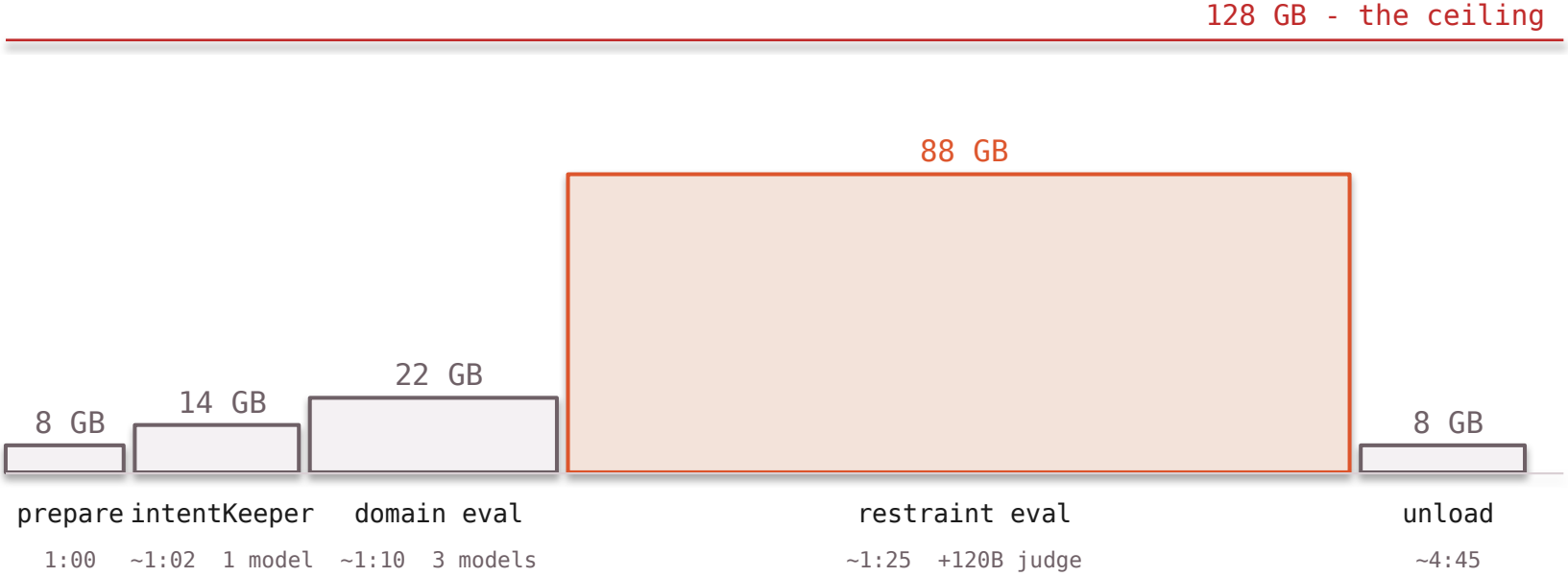
```
// common.ai calls the model - even a self-hosted one. It assumes it's already up.
Keeping it up is the gap.
```

SECTION 03

What it looks like every night.

// one run, quiet on purpose

One model set in memory at a time.



one eval's model set at a time - restraint's 88 GB never overlaps the flagship.

The same shape, in code.

```
prepare_box                          # free memory first
  >> wait >> intentkeeper            # wait = readiness sensor
  >> wait >> domain                  # one model set at a time
  >> wait >> restraint               #   + the 120B judge
  >> draft >> digest >> review >> file   # review = HUMAN GATE

# sensors: all_done    evals: all_success
```

```
# prepare_box clears whatever's loaded - the eval starts from a clean box.
```

The log is the discipline.



every model that ran, and when

every retry, every failure, in one place

which model ran, what it scored, what failed

```
// not a script that forgets by morning. A run history I can answer for.
```

Detection is solved. The response is the work.

domain · does it spot the crisis

97.8%

479 / 490 dataset d38138d9

restraint · does the reply hold

69.4%

340 / 490 dataset d38138d9

150 flagged. one, from the judge:

flag (conf 0.96): the reply refused, but never handed off to a crisis line.

```
# same corpus, same night. It knows it's a crisis - the reply is what still slips.
```

Tonight's run, parked at the gate.

[Airflow UI - the human gate, waiting]

one model at a time · the host stays up · the review waits for a tick

And honestly? It's boring.

```
// boring means it runs while I sleep, and every run explains itself by morning.
```

Five things, reusable on any exclusive workload.

- Treat the local model like a service, not a script
- Check readiness before inference, not after failure
- Give exclusive hardware one tenant at a time
- Separate model-output failure from infrastructure failure
- The run log is not optional for experimental stacks

Airflow is AI plumbing.

It's quiet, and it works.

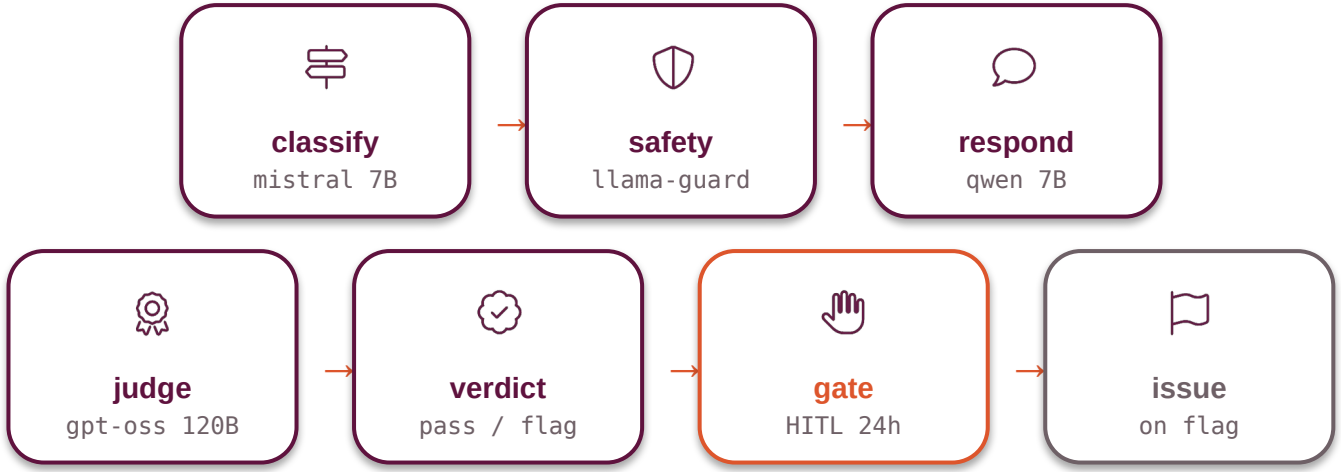
github.com/Olawoyin007

simplylocalai.com

THANK YOU



Appendix - one sample's path



■ model ■ you (the gate) ■ plain Airflow

row one's response is row two's input · built on Inspect (AISI).